

Instructivos relevantes

Cambios generales en comparación al Aux 2

Moví la clase Model a un archivo externo llamado `helpers.py`, donde también agregué la clase Mesh que se explica un poquito más abajo. Por ello, estoy importando este archivo al inicio del código:

```
import pygame
import numpy as np
import os
import trimesh as tm
from OpenGL import GL
from pathlib import Path
from utils.helpers import Model, Mesh # <-----
...
```

Si estan trabajando en una copia del repositorio, esto no afecta su código actual, pero si quieren que el código se vea mas limpio les recomiendo aplicarlo :)

Por otro lado, vamos a usar transparencia, así que hay que decirle a OpenGL que queremos que lo haga. Esto se llama “Blending” y para lograr el efecto de transparencia debemos llamar a estas funciones en el `on_draw()`:

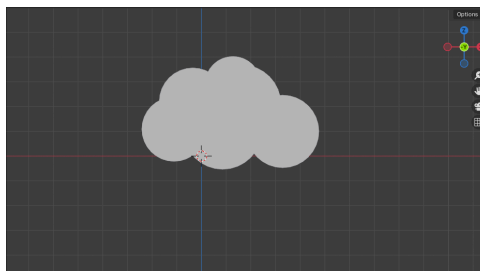
```
...
def on_draw():
    # Lineas para activar el uso del parámetro alpha
    GL.glEnable(GL.GL_DEPTH_TEST)
    GL.glEnable(GL.GL_BLEND)
    GL.glBlendFunc(GL.GL_SRC_ALPHA, GL.GL_ONE_MINUS_SRC_ALPHA)

    # color de fondo al limpiar un frame (0,0,0) es negro
    GL.glClearColor(1, 1, 1, 1.0)
    ...
```

Ya, ahora vayan a su carpeta CC3501 y hagan `git pull` para tener los ejemplos que van a salir en este documento :9

Hice algo en blender ¿Cómo lo abro con OpenGL?

Hice esta nube en blender:

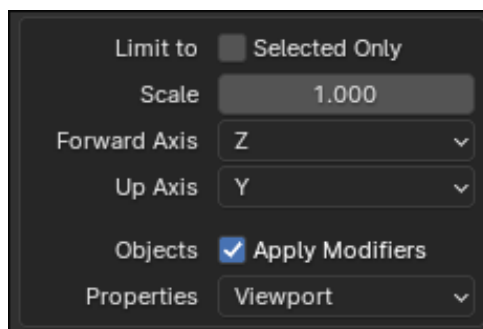


¿Cómo puedo tenerla en mi Tarea?

Exportar el modelo de blender

- Primero exportamos la nube como un obj. En Blender vamos a ir a `File > Export > Wavefront (.obj)`

- Van a ver una ventana con varias opciones. Fijense en donde dice cual va a ser el Forward Axis y el Up Axis:



Agregar al código

- Igual que con `Model()`, vamos a introducir una nueva clase llamada `Mesh()` que nos ayudará a trabajar con “meshes” o “mallas”.

Así declaramos un cloud:

```
cloud = Mesh(Path(os.path.dirname(__file__)) / 'assets/cloud.obj', 1, [1, 1, 1])
```

- El primer parámetro es el path hacia mi archivo obj
- El segundo es el alpha o transparencia de mi figura (va de 0 → 1)
- Y el tercero es el color base de mi figura
- Luego seguimos como en el Aux 2, hacemos `cloud.init_gpu_data(pipeline)`
- Y estamos ready :)

Este ejemplo está completo en el repo y se llama `blenderClouds.py`, pruebenlo y van a cachar como funciona.

Aquí también sucede algo que me preguntaron sobre `butterfly.py` ¿Cómo hago que la mariposa vuelva al inicio? Revisenlo quizás les sirve para lo que quieran lograr en su Tarea.

Quiero que mi figura tenga transparencia ¿Cómo lo hago?

Por si no cacharon, esto ya lo implementé en `blenderClouds.py`.

La diferencia con el aux 2 radica en 2 cosas:

1. Añadir la configuración que indiqué al inicio en `on_draw()`
2. Las implementaciones de `Model` y `Mesh` ahora incluyen un parámetro extra que es el **alpha** de la figura. Tomenlo en cuenta al crear un `Model` o un `Mesh`
3. Cambien a los shaders `transform_alpha.vert` y `color_alpha.frag`

En el repo se encuentra un ejemplo llamado `amongusTransparent.py` donde se dibujan varias figuras con distintas transparencias

Cualquier duda, sugerencia, reclamo o typo en este documento, denle al correo o foro („• ◡ •„)